

Cryptography Applications

3.1 Objectives

- Understand the application of symmetric and asymmetric cryptography.
- Demonstrate symmetric encryption and decryption using the Advanced Encryption Standard (AES).
- Demonstrate asymmetric encryption and decryption using the Rivest-Shamir-Adleman (RSA).
- Demonstrate different digital signature algorithms, including RSA Signature Scheme with Appendix – Probabilistic Signature Scheme (RSASSA-PSS, ak.a., PKCS#1 PSS) and Elliptic Curve Digital Signature Algorithm (ECDSA).

3.2 Introduction

In this practicum, students will demonstrate and evaluate the usage of different cryptography schemes using the provided Jupyter Notebook in GitLab. By making use of the already provided and implemented functions, the student will be communicating with another student using an unsecure communication channel. To simplify the exchange of messages between students, this practicum uses JSONbin.io website and its public API. Keep in mind that this is for real an unsecure communication channel given that ANY message published on the website can be read by ANYONE on the public internet.

Steps to use JSONbin.io API:

- Create an account and sign in to JSONbin.io
- From your JSONbin.io dashboard, under the "API KEYS" tab, you can obtain your personal X-Master-Key that is used to access the API. This key is unique to each JSONbin.io account and should be kept private.
- From your JSONbin.io dashboard, under the "BINS" tab, you can create a new PUBLIC bin. You can insert the following JSON to successfully initialize it: {"Lab":"Crypto"}. Note: Remember to set the bin as public.
- Save your BIN ID. This will be your "unsecure upload communication channel". Your published message will be saved in this BIN.

- Ask your friend to provide their BIN ID. This will be the "unsecure download communication channel" to be used for receiving the messages from your friend.

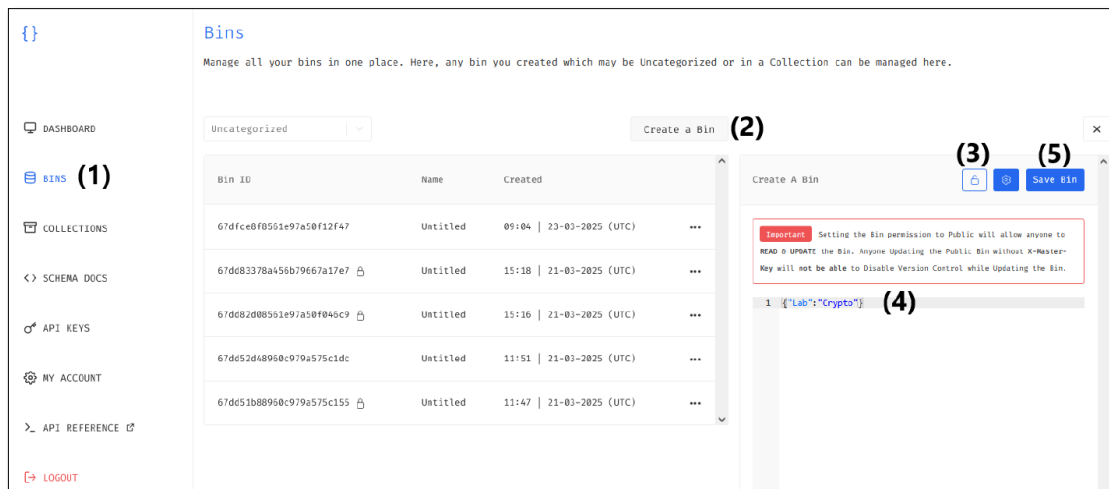


Figure 2.1: JSONbin.io step-by-step guide.

Utility Functions:

In the "Utility functions" chapter, you can find all the functions to be used to upload/download messages to JSONbin, encrypt/decrypt messages using AES/RSA, generate RSA/EC keys, and generate/validate digital signatures using PKCS#1 PSS and ECDSA. Also, examples on how to use these functions are provided. The code in the "Utility functions" chapter is not meant to be modified; it is there to help you complete the tasks in the "Assignment" chapter.

3.3 Encryption and Decryption using AES

Symmetric cryptography, also known as secret-key cryptography, is a method in which a single pre-shared key is employed for both encryption and decryption of data. This approach relies heavily on the confidentiality of the key itself, as anyone possessing the key can access and interpret the encrypted information. Due to this requirement, entities utilizing symmetric cryptography must securely exchange and manage the secret key prior to any communication. This initial exchange often presents practical challenges, as it requires a secure channel or prior trust between parties to prevent unauthorized interception.

One of the primary strengths of symmetric cryptography is its efficiency. Algorithms utilizing symmetric keys, such as the Advanced Encryption Standard (AES), are computationally faster and require fewer resources compared to their asymmetric counterparts. AES is particularly noteworthy and widely adopted globally due to its robust security characteristics. Chosen by the U.S. National Institute of Standards and Technology (NIST), AES supports various key lengths, most commonly 128, 192, or 256 bits, and modes of operations, allowing users to select a security level appropriate for their needs.

Symmetric cryptography is extensively utilized in scenarios where speed and performance are critical, such as encrypting large volumes of data or securing communication channels within

controlled environments. Common use cases include securing data at rest, such as files stored on a hard drive, or encrypting data in transit over internal network systems, like those within corporations or government institutions. However, the primary challenge remains the secure exchange of keys, which often requires additional cryptographic protocols to facilitate safe key management.

In the following, a brief description of the utility functions implemented in the Jupyter notebook and meant to be used to complete the assignment is provided. More details about the required inputs and expected outputs can be found in the Jupyter notebook.

Available functions:

- `encrypt(message, passphrase, key_length, mode) → (nonce/iv +) encrypted_message (+ tag)`
- `decrypt(ciphertext, passphrase, key_length, mode) → message`

Supported modes of operation:

- Electronic Codebook Mode (ECB)
- Cipher Block Chaining Mode (CBC)
- Cipher Feedback Mode (CFB)
- Output Feedback Mode (OFB)
- Counter Mode (CTR)
- Counter with CBC-MAC mode (CCM)
- Galois/Counter Mode (GCM)
- Offset Codebook Mode (OCB)

Supported key length:

- 128
- 192
- 256

Symmetric encryption assignment

In this part, the student is expected to experiment using AES cryptographic algorithm based on the provided function in the Jupyter notebook. The students are expected to work in pair to exchange encrypted message through JSONbin.io. The provided implementation is used to answer the assignment questions. Students will explore AES encryption by performing the following tasks:

1. Encrypt a message: Use AES with a chosen mode of operation and key length to encrypt a given message using a pre-shared secret key.
2. Publish to JSONbin: Upload the encrypted message to JSONbin.
3. Retrieve and Decrypt: Fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct passphrase, key length, and mode of operation.
4. Analyze AES-GCM Behaviour: Encrypt and decrypt the same message multiple times using AES-GCM-128 and explain why the ciphertext varies each time.

5. Test Mode & Key Length Variations: encrypt a message using six different combinations of AES modes and key lengths, then analyze how these factors affect ciphertext length.
6. Discussion: Briefly explain the impact of mode of operation selection, key length, IV/nonce, and authentication tags on ciphertext size and security.

3.4 Encryption and Decryption using RSA

Asymmetric cryptography, also known as public-key cryptography, uses two mathematically linked keys: a public key (which is shared openly) and a private key (which is kept secret). Unlike symmetric cryptography, which relies on a single shared key for encryption and decryption, asymmetric systems enable secure communication by exchanging only the public keys among communicating entities. The public key encrypts data or verifies digital signatures, while the private key decrypts or creates signatures. This separation allows secure messaging (e.g., email encryption) and authentication (e.g., digital certificates), forming the backbone of modern internet security protocols like Transport Layer Security (TLS)/ Secure Sockets Layer (SSL).

The RSA algorithm, a pioneering asymmetric method, relies on the computational difficulty of factoring large prime numbers. To generate keys, two large primes p and q are multiplied to create a modulus $n = p \times q$. Using Euler's totient function $\phi(n) = (p-1)(q-1)$, a public exponent e is chosen such that it is coprime to $\phi(n)$. The private exponent d is computed as $d \equiv e^{-1} \bmod \phi(n)$. Encryption involves raising a message m to e and applying the modulo operation with divisor equal to n . On the contrary, decryption involves raising the encrypted message c to d , and, as before, applying the modulo operation with divisor equal to n . Security hinges on the infeasibility of factoring n to derive p and q , which would expose d . RSA remains foundational for secure data transmission, digital signatures, and key exchange in hybrid systems (e.g., combining RSA with symmetric AES for efficiency). Despite requiring longer keys (2048+ bits) to resist modern factorization attacks, its simplicity and reliability ensure its use in HTTPS, VPNs, and blockchain.

In the following, a brief description of the utility functions implemented in the Jupyter notebook and meant to be used to complete the assignment, is provided. More details about the required inputs and expected outputs can be found in the Jupyter notebook.

Available functions:

- `generate_rsa_keys(bits) → private_key, public_key, rsa_components`
- `save_rsa_keys(private_key, public_key, private_key_filename, public_key_filename) → None`
- `import_rsa_key(key_filename) → RSA key object`
- `read_rsa_key(key_string) → RSA key object`
- `encrypt_rsa(message, public_key) → encrypted_message`
- `decrypt_rsa(encrypted_message, private_key) → message`
- `sign_message_pss(message, private_key) → signature`
- `verify_signature(message, signature, public_key) → True/False`

Supported RSA key sizes (in bits):

- 1024

- 2048
- 3072
- 4096

Asymmetric encryption assignment

In this part, the student is expected to experiments using RSA cryptographic algorithm based on the provided functions in the Jupyter notebook. The students are expected to work in pair to exchange encrypted messages through JSONbin.io. The provided implementation is used to answer the assignment questions. Students will work with RSA encryption by performing the following tasks:

1. Generate and save RSA Key Pairs: Create a new pair of RSA private/public keys with a selected key length and save them.
2. Print Keys: Display the generated private and public keys in a readable format.
3. Import RSA Keys: Load previously saved RSA keys from files for encryption and decryption.
4. Encrypt a Message: Request a friend's public key, use it to encrypt a confidential message, and publish the ciphertext to JSONbin.
5. Retrieve and Decrypt: Fetch an encrypted message previously published by your friend on JSONbin and decrypt it using the correct private key.
6. Experiment with Key Lengths: Generate RSA keys with different key lengths (1024, 2048, 3072, 4096 bits) and analyze the key components (n, e, d, p, q).
7. Discussion: Briefly compare how key length impacts security, performance, and ciphertext size.

3.5 Elliptic Curves Digital Signature Algorithm (ECDSA)

The Elliptic Curve Digital Signature Algorithm (ECDSA) is a cryptographic protocol used to generate and verify digital signatures, leveraging the mathematical properties of elliptic curves over finite fields. As an evolution of the Digital Signature Algorithm (DSA), ECDSA replaces traditional modular arithmetic with elliptic curve operations, enabling stronger security with shorter key lengths compared to RSA or DSA. For instance, a 256-bit ECDSA key offers security comparable to a 3072-bit RSA key. This efficiency makes ECDSA ideal for resource-constrained environments like mobile devices, blockchain systems (e.g., Bitcoin and Ethereum), and IoT applications. Its adoption is widespread in protocols such as TLS, SSH, and digital identity frameworks, where both speed and compact signature sizes are critical.

ECDSA operates by generating a private key (a random integer) and a corresponding public key (a point on the curve derived by multiplying the private key with a predefined base point). To sign a message, the sender hashes the data and computes a signature using their private key and a cryptographically secure random nonce. The signature consists of two components, (r, s) , generated through elliptic curve point multiplication and modular operations. Verification involves recalculating the curve point using the sender's public key, the signature, and the message hash,

confirming it matches the original r value. Security relies on the elliptic curve discrete logarithm problem (ECDLP) with computational infeasibility of deriving the private key from the public key.

In the following, a brief description of the utility functions implemented in the jupyter-notebook and meant to be used to complete the assignment is provided. More details about the required inputs and expected outputs can be found in the jupyter-notebook.

Available functions:

- `generate_ecc_keys(curve) → private_key, public_key`
- `import_ecc_key(key_filename) → ECC key object`
- `read_ecc_key(key_string) → ECC key object`
- `export_ecc_key(key) → None`
- `sign_message_ecc(message, private_key) → signature`
- `verify_signature_ecc(message, signature, public_key) → True/False`

Signature curves:

- NIST P-192
- NIST P-224
- NIST P-256
- NIST P-384
- NIST P-521
- Ed25519
- Ed448

Digital signatures assignment

In this part, the student is expected to experiment using RSASSA-PSS and ECDSA cryptographic algorithms based on the provided function in the Jupyter notebook. The implementation is used to answer the assignment questions. The students are expected to work in pair to exchange encrypted and signed messages through JSONbin.io. Students will explore digital signatures using RSASSA-PSS and ECDSA. They will complete the following tasks:

1. Sign and Encrypt a Message: Encrypt a message using RSA, generate a digital signature, and publish both to JSONbin.
2. Retrieve and Verify: Fetch an encrypted message and its signature from JSONbin, verify its authenticity using the sender's public key, and decrypt it.
3. Key Usage Discussion: Explain which keys were used for encryption/decryption versus signature generation/validation.
4. Signature Experiments: Generate and compare signatures using different key types (e.g., RSA 1024-bit and 4096-bit, ECDSA NIST P-192, P-224, and P-521).
5. Key & Signature Analysis: Briefly discuss how key type and length affect signature size, security, and performance in RSA vs. ECC.

3.6 Assignment

In this assignment, each student must write an individual report based on the Jupyter notebook implementation.

Answer the following questions in your report, provide the practicum results using the saved data and images to explain your answers, and follow the Report Guidelines to write the report.

1. Symmetric encryption:
 - a. Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - b. Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - c. Briefly discuss how different modes of operation and key lengths affect ciphertext length and security.
2. Asymmetric encryption:
 - a. Describe the various steps you followed to encrypt, upload, download, and decrypt the secret messages.
 - b. Discuss which information you needed to share with your friend to enable the exchange of the encrypted messages.
 - c. Regarding the previous point, compare with what was required when using symmetric encryption. What are the pros and cons of each cryptography scheme?
 - d. How does the RSA key length relate to the length (in bits) of the RSA components (i.e., n, p, q, e, d)?
3. Digital signatures:
 - a. Describe the various steps you followed to encrypt, sign, upload, download, decrypt, and verify the secret signed messages. In particular, highlight which private and public keys were used for each operation.
 - b. Briefly discuss and compare (qualitatively) how PKCS#1 PSS and ECDSA differ in terms of: private and public key length, signature size, security, and performance